

第5章 OpenMP共享内存编程

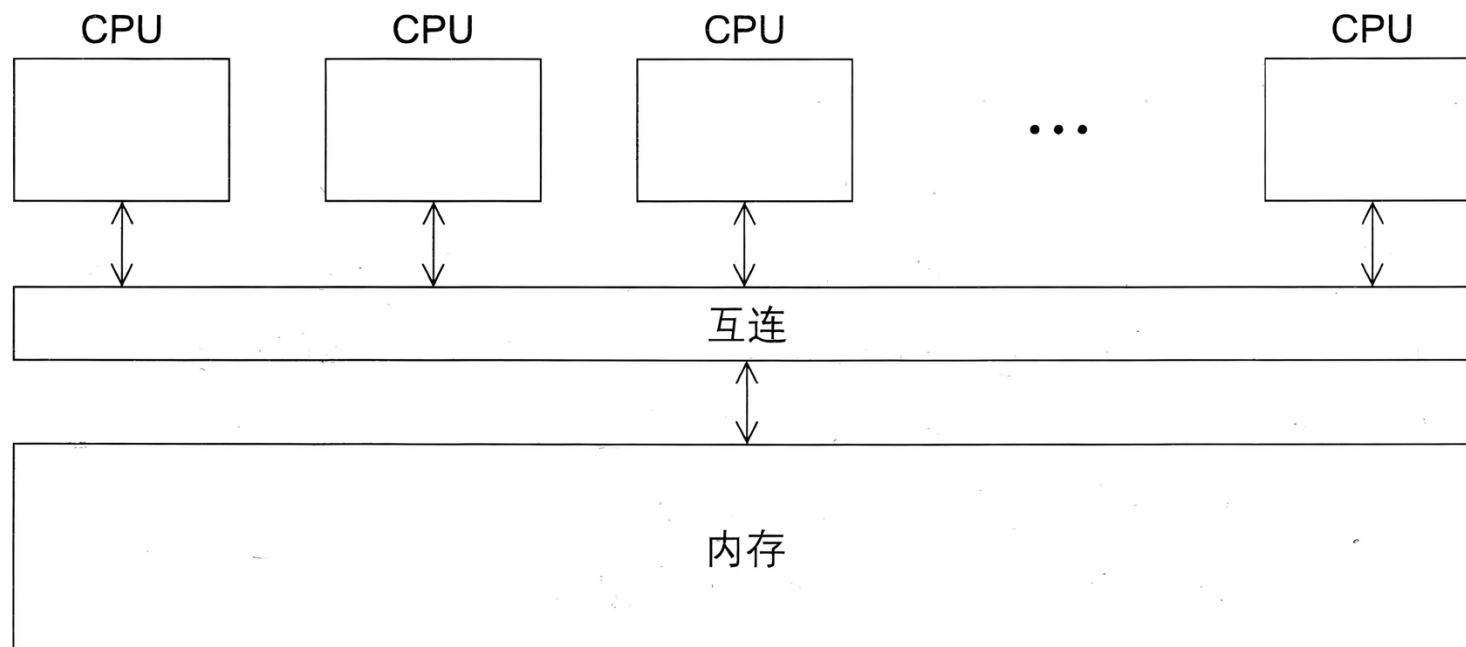
计算机与软件学院 陆克中

路线图

- 编写使用OpenMP的程序
- 仅对源代码做少量修改，使用OpenMP对多个串行for循环进行并行化
- 任务并行
- 显式线程同步
- 共享内存编程中的典型问题

OpenMP

- 用于共享内存并行编程的API。
- MP = 多处理（multiprocessing）。
- 专为共享内存系统设计。
 - ▶ 由若干核心（CPU）组成的集合，它们均可访问主内存。



编译指示

- 特殊的预处理器指令。
- 通常被添加到系统中，以允许执行不属于基本C规范的行为。
- 不支持pragma的编译器会直接忽略它们。

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void Hello(void); /* 线程函数 */
int main(int argc, char* argv[]) {
    /* 从命令行中获得线程的数量 */
    int thread_count = atoi(argv[1]);
    #pragma omp parallel num_threads(thread_count)
    Hello();
    return 0;
}
void Hello(void) {
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    printf("Hello from thread %d of %d\n", my_rank,
thread_count);
}
```

编译

■ Windows

- ▶ Visual Studio项目->属性
 - C/C++->语言->OpenMP支持 (Release和Debug配置分别设定)

■ Linux

- ▶ C: gcc -O3 -fopenmp -o a.out a.c
- ▶ C++: g++ [-O3] [-g] [-Wall] -fopenmp -o a.out a.cc

编译器优化 生成调试信息 开启编译器警告 启用OpenMP



The diagram consists of four arrows pointing from the labels below to the flags in the C++ command line above. The first arrow points from '编译器优化' to '-O3'. The second arrow points from '生成调试信息' to '-g'. The third arrow points from '开启编译器警告' to '-Wall'. The fourth arrow points from '启用OpenMP' to '-fopenmp'.

运行

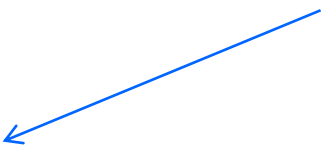
`./omp_hello 4`

4个线程运行



可能的输出

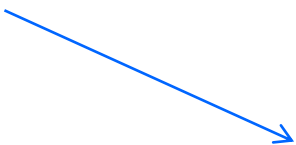
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4



Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4



Hello from thread 3 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4



OpenMP编译指示

■ #pragma omp parallel

- ▶ 最基本的并行指令。
- ▶ 运行后续结构化代码块的线程数量由运行时系统决定。



一个进程分叉并加入两个线程

子句

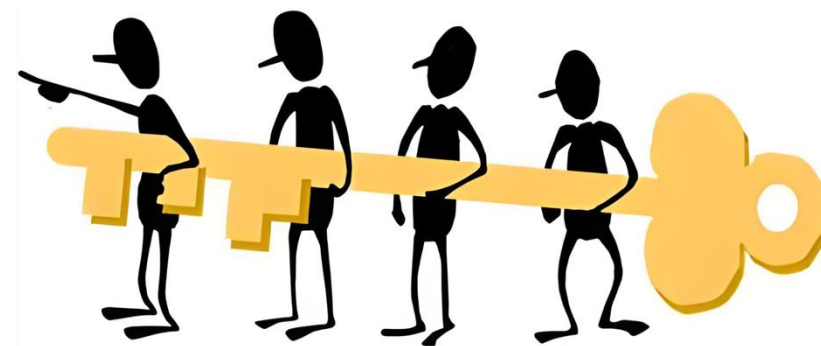
- `#pragma omp parallel num_threads(thread_count)`
 - ▶ 用于修改指令的文本。
 - ▶ 可以将num_threads子句添加到并行指令中。
 - ▶ 它允许程序员指定执行后续代码块所需的线程数量。

值得注意的是.....

- 程序能够启动的线程数量可能存在系统定义的限制。
- OpenMP标准并不保证实际会启动`thread_count`个线程。
- 当前大多数系统可以启动数百甚至数千个线程。
- 除非我们尝试启动大量线程，否则几乎总能获得期望数量的线程。

一些术语

- 用OpenMP的术语来说，执行并行代码块的线程集合（包括初始线程和新建线程）称为**线程组**，初始线程称为**主线程**（master），额外创建的线程称为**从线程**（slaves）。



若编译器不支持 OpenMP

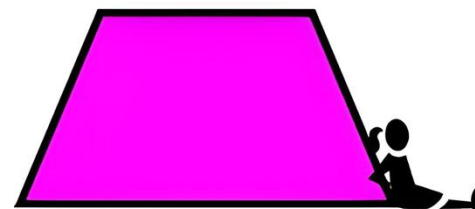
```
#include <omp.h>
```



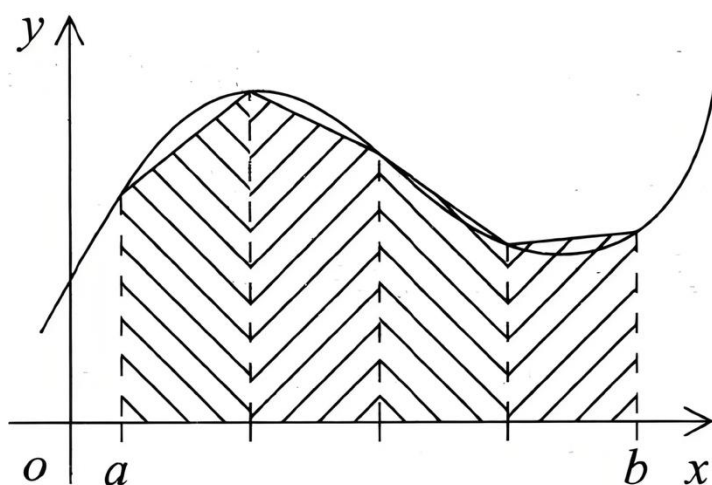
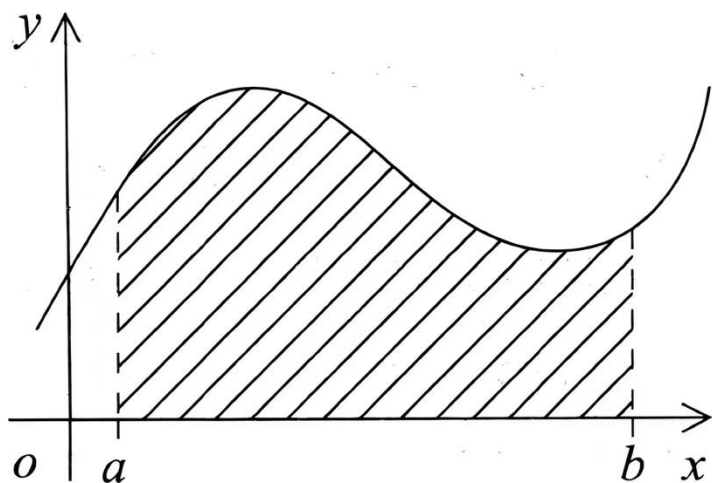
```
#ifdef _OPENMP  
#include <omp.h>  
#endif
```

```
#ifdef _OPENMP  
int my_rank = omp_get_thread_num();  
int thread_count = omp_get_num_threads();  
#else  
int my_rank = 0;  
int thread_count = 1;  
#endif
```

梯形法则



梯形法则



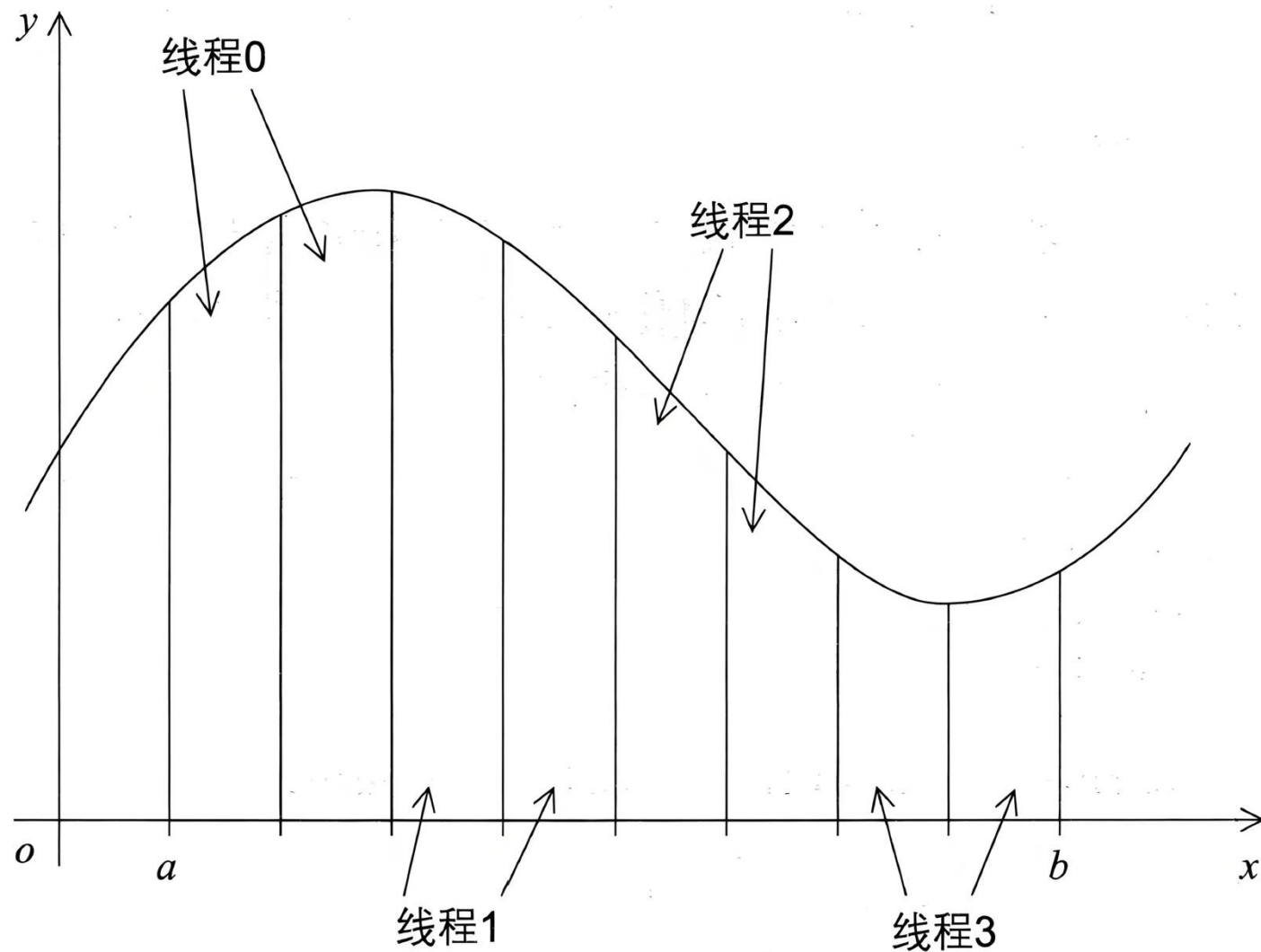
```
/* 输入: a, b, n */  
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
for (i = 1; i <= n-1; i++) {  
    x_i = a + i*h;  
    approx += f(x_i);  
}  
approx = h*approx;
```

串行算法

第一个OpenMP版本

- 我们确定了两种类型的工作：
 - a) 计算各个梯形的面积;
 - b) 梯形的面积相加。
- 第一个集合中的工作之间没有通信，但第一个集合中的每个工作都需要与工作b进行通信。
- 假设梯形比核心多得多。
- 所以我们通过给每个线程分配一个连续的梯形块（每个核心分配一个线程）来汇集工作。

为线程分配梯形



竞争条件

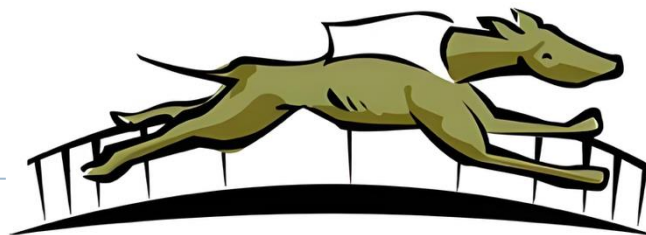
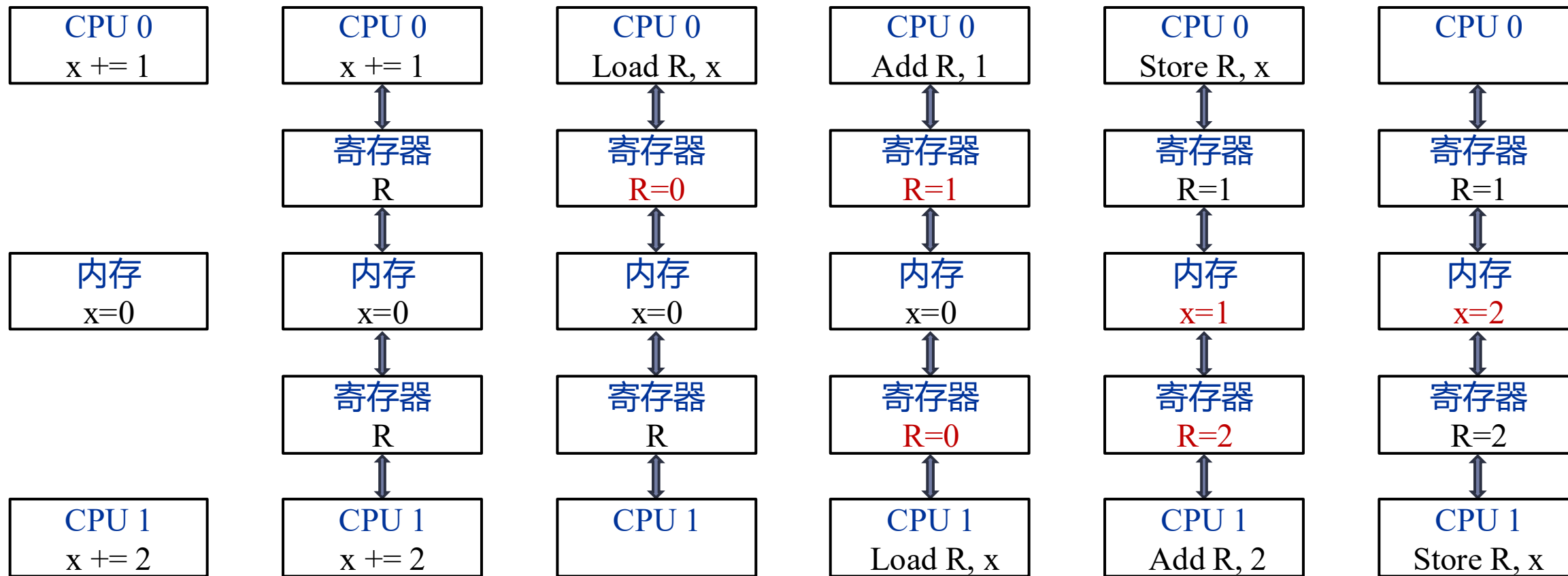
- 当两个（或更多）线程试图同时执行时，会产生不可预测的结果：

`global_result += my_result ;`

时间	线程 0	线程 1
0	将 <code>global_result = 0</code> 写进寄存器	完成 <code>my_result</code>
1	将 <code>my_result = 1</code> 写进寄存器	将 <code>global_result = 0</code> 写进寄存器
2	将 <code>my_result</code> 加进 <code>global_result</code>	将 <code>my_result = 2</code> 写进寄存器
3	存储 <code>global_result = 1</code>	将 <code>my_result</code> 加进 <code>global_result</code>
4		存储 <code>global_result = 2</code>



竞争条件



互斥

```
#pragma omp critical  
global_result += my_result ;
```



每次只有一个线程可以执行下面的结构化代码块。

首个OpenMP梯形法则程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
double f(double x) {
    double return_val;
    //return_val = x * x;
    return_val = 4 / (1 + x * x);
    return return_val;
}
void Trap(double a, double b, int n,
double* global_result_p) {
    double h, x, my_result;
    double local_a, local_b;
    int i, local_n;
    int my_rank =
omp_get_thread_num();
    int thread_count =

omp_get_num_threads();
    h = (b - a) / n;
    local_n = n / thread_count;
    local_a = a + my_rank * local_n
* h;
    local_b = local_a + local_n * h;
    my_result = (f(local_a) +
f(local_b)) / 2.0;
    for (i = 1; i <= local_n - 1; i++) {
        x = local_a + i * h;
        my_result += f(x);
    }
    my_result = my_result * h;
#pragma omp critical
    * global_result_p += my_result;
}
int main(int argc, char* argv[]) {
    double global_result = 0.0;

double a, b;
int n;
int thread_count = 4;
if (argc > 1)
    thread_count = atoi(argv[1]);
printf("Enter a, b, and n\n");
scanf_s("%lf %lf %d", &a, &b,
&n);
#pragma omp parallel
num_threads(thread_count)
    Trap(a, b, n, &global_result);
    printf("With n = %d trapezoids,
our estimate\n", n);
    printf("of the integral from %f to
%f = %.14e\n", a, b, global_result);
    return 0;
}
```

变量的作用域



作用域

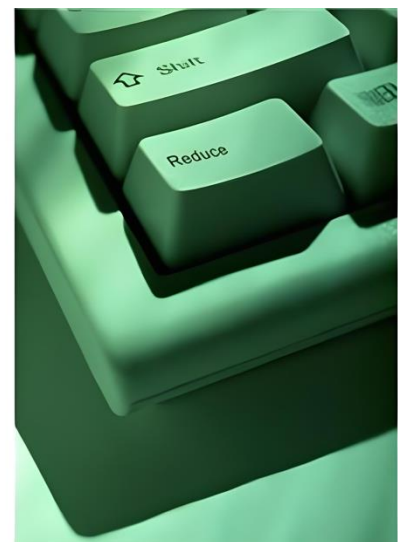
- 在串行编程中，变量的作用域是指程序中可以使用该变量的那些部分。
- 在OpenMP中，变量的作用域是指在一个并行块中可以访问该变量的线程集合。

OpenMP中的作用域

- 能够被线程组中所有线程访问的变量具有**共享**作用域。
- 只能被单个线程访问的变量具有**私有**作用域。
- 在并行块之前声明的变量，其默认作用域为**共享**。



归约子句



OpenMP梯形法则程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
double f(double x) {
    double return_val;
    //return_val = x * x;
    return_val = 4 / (1 + x * x);
    return return_val;
}
double Local_trap(double a, double
b, int n) {
    double h, x, my_result;
    double local_a, local_b;
    int i, local_n;
    int my_rank =
omp_get_thread_num();
    int thread_count =
omp_get_num_threads();
```

```
h = (b - a) / n;
local_n = n / thread_count;
local_a = a + my_rank * local_n
* h;
local_b = local_a + local_n * h;
my_result = (f(local_a) +
f(local_b)) / 2.0;
for (i = 1; i <= local_n - 1; i++) {
    x = local_a + i * h;
    my_result += f(x);
}
my_result = my_result * h;
return my_result;
}
int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;
    int n;
    int thread_count = 4;
```

```
if (argc > 1)
    thread_count = atoi(argv[1]);
printf("Enter a, b, and n\n");
scanf_s("%lf %lf %d", &a, &b,
&n);
#pragma omp parallel
num_threads(thread_count)
{
    #pragma omp critical
    global_result += Local_trap(a,
b, n);
}
printf("With n = %d trapezoids,
our estimate\n", n);
printf("of the integral from %f to
%f = %.14e\n", a, b, global_result);
return 0;
}
```


全局和

- 强迫线程按顺序执行。

```
    global_result = 0.0;
#   pragma omp parallel num_threads(thread_count)
    {
#       pragma omp critical
        global_result += Local_trap(double a, double b, int n);
    }
```

全局和

- 可以通过在并行块内声明一个私有变植并将临界区移到函数调用之后来避免这个问题。

```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count)
{
    double my_result = 0.0; /* 私有的 */
    my_result += Local_trap(double a, double b, int n);
# pragma omp critical
    global_result += my_result;
}
```



归约运算符

- **归约运算符**是一个可结合的二元运算（比如加法或乘法）。
- **归约**是将同一个归约运算符重复应用于一连串的操作数以得到一个结果的计算。
- 操作的所有中间结果都应该存储在同一个变量中：归约变量。

归约运算符

- 可以在并行指令中加入一个归约子句。

`reduction(<operator>: <variable list>)`

 `+, *, -, &, |, ^, &&, ||`

```
global_result = 0.0;
# pragma omp parallel num_threads(thread_count) \
    reduction(+: global_result)
global_result += Local_trap(double a, double b, int n);
```

归约运算

■ 并行域开始

- ▶ 给每个线程创建变量私有副本，并用对应算子初始化。

■ 各线程并行计算

- ▶ 每个线程只改自己的私有副本，无数据竞争、不用加锁。

■ 并行循环结束

- ▶ 所有线程私有副本，按二叉树树形归约逐级合并。

■ 结果写回全局变量

- ▶ 合并后的最终值赋值给原始全局变量。

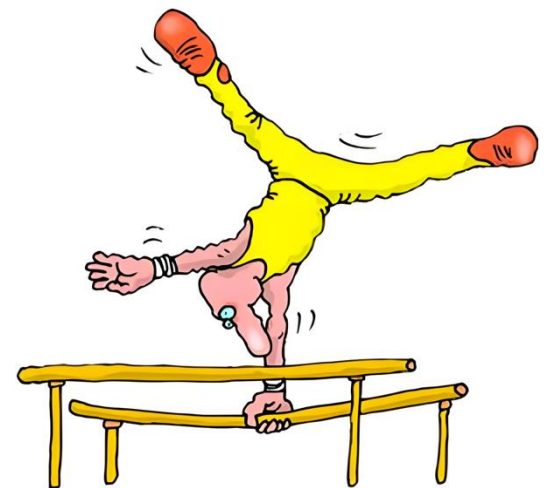
OpenMP梯形法则程序

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
double f(double x) {
    double return_val;
    //return_val = x * x;
    return_val = 4 / (1 + x * x);
    return return_val;
}
double Local_trap(double a, double
b, int n) {
    double h, x, my_result;
    double local_a, local_b;
    int i, local_n;
    int my_rank =
omp_get_thread_num();
    int thread_count =
```

```
omp_get_num_threads();
    h = (b - a) / n;
    local_n = n / thread_count;
    local_a = a + my_rank * local_n
* h;
    local_b = local_a + local_n * h;
    my_result = (f(local_a) +
f(local_b)) / 2.0;
    for (i = 1; i <= local_n - 1; i++) {
        x = local_a + i * h;
        my_result += f(x);
    }
    my_result = my_result * h;
    return my_result;
}
int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;
```

```
int n;
int thread_count = 4;
if (argc > 1)
    thread_count = atoi(argv[1]);
printf("Enter a, b, and n\n");
scanf_s("%lf %lf %d", &a, &b,
&n);
#pragma omp parallel
num_threads(thread_count)
reduction(+: global_result)
    global_result += Local_trap(a, b,
n);
printf("With n = %d trapezoids,
our estimate\n", n);
printf("of the integral from %f to
%f = %.14e\n", a, b, global_result);
return 0;
}
```

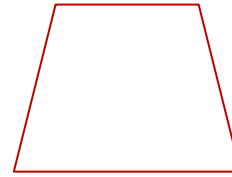
PARALLEL FOR指令



parallel for

- 分叉一组线程来执行下面的结构块。
- 然而，在parallel for指令之后的结构化块必须是一个for循环。
- 此外，通过parallel for指令，系统通过在线程之间分配循环的迭代来实现for循环的并行化。

```
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
for (i = 1; i <= n-1; i++)  
    approx += f(a + i*h);  
approx = h*approx;
```



```
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
# pragma omp parallel for num_threads(thread_count) \  
    reduction(+: approx)  
for (i = 1; i <= n-1; i++)  
    approx += f(a + i*h);  
approx = h*approx;
```

语句并行化的合法形式

for $\left(\begin{array}{l} \text{index} = \text{start} \quad ; \quad \text{index} \geq \text{end} \quad ; \quad \text{index} += \text{incr} \\ \text{index} < \text{end} \quad \text{index}++ \\ \text{index} \leq \text{end} \quad ++\text{index} \\ \text{index} > \text{end} \quad \text{index}- \\ \text{index} > \text{end} \quad -\text{index} \\ \text{index} = \text{index} + \text{incr} \\ \text{index} = \text{incr} + \text{index} \\ \text{index} = \text{index} - \text{incr} \end{array} \right)$

注意事项

- 变量`index`必须是整型或指针类型（例如，不能是浮点型）。
- 表达式`start`、`end`和`incr`必须有一个兼容的类型。例如，如果`index`是指针，那么`incr`必须具有整数类型。
- 在循环的执行过程中，表达式`start`、`end`和`incr`不能改变。
- 在循环的执行过程中，变量`index`只能被`for`语句中的“增加表达式”修改。

数据依赖性

```
fibonacci[0] = fibonacci[1] = 1;
for (i = 2; i < n; i++)
    fibonacci[i] = fibonacci[i - 1] + fibonacci[i - 2];
```



```
fibonacci[0] = fibonacci[1] = 1;
#pragma omp parallel for num_threads(2)
for (i = 2; i < n; i++)
    fibonacci[i] = fibonacci[i - 1] + fibonacci[i - 2];
```



1 1 2 3 5 8 13 21 34 55

这是正确的



1 1 2 3 5 8 0 0 0 0

但有时会出现这种情况

```
#include <iostream>
#include <omp.h>
using namespace std;
const int n = 20;
int main()
{
    int i;
    int fibonacci[n];
    fibonacci[0] = fibonacci[1] = 1;
    #pragma omp parallel for num_threads(4)
    private(i)
    for (i = 2; i < n; i++)
        fibonacci[i] = fibonacci[i - 1] + fibonacci[i - 2];
    for (i = 0; i < n; i++)
        cout << fibonacci[i] << endl;
}
```

发生了什么事？



- 对于使用parallel for指令进行并行化的循环，OpenMP编译器不会检查循环迭代之间的数据依赖关系。
- 如果一个循环中，一次或多次迭代的结果依赖于其他迭代，那么通常情况下，OpenMP无法对其进行正确并行化。

估算 π

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots + (-1)^n \frac{1}{2n+1} + \cdots \right)$$

```
double factor = 1.0;
double sum = 0.0;
for (i = 0; i < n; i++, factor = -factor) {
    sum += factor/(2*i+1);
}
pi = 4.0*sum;
```

OpenMP解决方案#1

```
#include <iostream>
#include <iomanip>
#include <omp.h>
using namespace std;
int main(){
    int k;
    int n = 1000000000;
    double factor = 1.0;
    double sum = 0.0;
    #pragma omp parallel for
    num_threads(2) reduction(+:sum)
    for (k = 0; k < n; k++) {
        sum += factor / (2 * k + 1);
        factor = -factor;
    }
    double pi_approx = 4.0 * sum;
    cout << setprecision(15) << pi_approx
    << endl;
}
```


OpenMP解决方案#1

循环依赖

```
# double factor = 1.0;
double sum = 0.0;
#pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum)
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

OpenMP解决方案#2

```
#include <iostream>
#include <iomanip>
#include <omp.h>
using namespace std;
int main(){
    int k;
    int n = 1000000000;
    double factor = 1.0;
    double sum = 0.0;
    #pragma omp parallel for
    num_threads(4) reduction(+:sum)
    private(factor)
    for (k = 0; k < n; k++) {
        if (k % 2 == 0)
            factor = 1.0;
        else
            factor = -1.0;
        sum += factor / (2 * k + 1);
    }
    double pi_approx = 4.0 * sum;
    cout << setprecision(15) << pi_approx
    << endl;
}
```

OpenMP解决方案#2

```
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum) private(factor)
for (k = 0; k < n; k++) {
    if (k % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;
    sum += factor / (2 * k + 1);
}
```



确保factor具有
私有作用域。

私有变量的初始化

```
#include <iostream>
#include <omp.h>
using namespace std;
int main() {
    int x = 5;
    #pragma omp parallel num_threads(4) private(x)
    {
        int my_rank = omp_get_thread_num();
        printf("Thread %d > before initialization, x = %d\n", my_rank, x);
        x = 2 * my_rank + 2;
        printf("Thread %d > after initialization, x = %d\n", my_rank, x);
    }
    printf("After parallel block, x = %d\n", x);
}
```

default子句

- 允许程序员指定代码块中每个变量的作用域。

default(none)

- 借助该子句，编译器会要求我们明确指定在代码块中使用、但在代码块外部声明的每个变量的作用域。

```
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    default(none) reduction(+:sum) private(k, factor) \
    shared(n)
for (k = 0; k < n; k++) {
    if (k % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;
    sum += factor/(2*k+1);
}
```

关于OpenMP中循环的更多内容：排序



冒泡排序

```
for (list_length = n; list_length >= 2; list_length--)  
    for (i = 0; i < list_length - 1; i++)  
        if (a[i] > a[i+1]) {  
            tmp = a[i];  
            a[i] = a[i+1];  
            a[i+1] = tmp;  
        }
```



串行奇偶移项排序

```
for (phase = 0; phase < n; phase++)  
    if (phase % 2 == 0)  
        for (i = 1; i < n; i += 2)  
            if (a[i-1] > a[i]) Swap(&a[i-1], &a[i]);  
    else  
        for (i = 1; i < n-1; i += 2)  
            if (a[i] > a[i+1]) Swap(&a[i], &a[i+1]);
```

阶段	数组下标			
	0	1	2	3
0	9	↔	7	8 ↔ 6
	7	9	6	8
1	7	9 ↔ 6	8	
	7	6	9	8
2	7 ↔ 6	9	↔ 8	
	6	7	8	9
3	6	7 ↔ 8	9	
	6	7	8	9

奇偶排序的第一个OpenMP实现

```
#include <stdio.h>
#include <stdlib.h>
#include <utility>
#include <omp.h>
const int RMAX = 10000000;
int thread_count = 4;
int main(int argc, char* argv[]) {
    int n = 100000;
    double start, finish;
    if (argc > 1)
        thread_count =
atoi(argv[1]);
    int* a = new int[n];
    int phase, i;
    srand(1);
    for (i = 0; i < n; i++)
        a[i] = rand() % RMAX;
```

```
    start = omp_get_wtime();
    for (phase = 0; phase < n;
phase++) {
        if (phase % 2 == 0)
            #pragma omp parallel for
num_threads(thread_count)
default(none) shared(a, n)
private(i)
                for (i = 1; i < n; i += 2)
                {
                    if (a[i - 1] > a[i])
                        std::swap(a[i - 1],
a[i]);
                }
            else
                #pragma omp parallel for
num_threads(thread_count)
```

```
default(none) shared(a, n)
private(i)
                for (i = 1; i < n - 1; i += 2)
                {
                    if (a[i] > a[i + 1])
                        std::swap(a[i], a[i +
1]);
                }
            }
    finish = omp_get_wtime();
    printf("Elapsed time = %e
seconds\n", finish - start);
    delete[] a;
    return 0;
}
```

奇偶排序的第二个OpenMP实现

```
#include <stdio.h>
#include <stdlib.h>
#include <utility>
#include <omp.h>
const int RMAX = 10000000;
int thread_count = 4;
int main(int argc, char* argv[]) {
    int n = 100000;
    double start, finish;
    if (argc > 1)
        thread_count =
atoi(argv[1]);
    int* a = new int[n];
    int phase, i;
    srand(1);
    for (i = 0; i < n; i++)
```

```
    a[i] = rand() % RMAX;
    start = omp_get_wtime();
    #pragma omp parallel
    num_threads(thread_count)
    default(none) shared(a, n)
    private(i, phase)
        for (phase = 0; phase < n;
phase++) {
            if (phase % 2 == 0)
                #pragma omp for
                for (i = 1; i < n; i += 2) {
                    if (a[i - 1] > a[i])
                        std::swap(a[i - 1],
a[i]);
                }
            else
```

```
                #pragma omp for
                for (i = 1; i < n - 1; i += 2)
                {
                    if (a[i] > a[i + 1])
                        std::swap(a[i], a[i +
1]);
                }
            }
    finish = omp_get_wtime();
    printf("Elapsed time = %e
seconds\n", finish - start);
    delete[] a;
    return 0;
}
```

性能

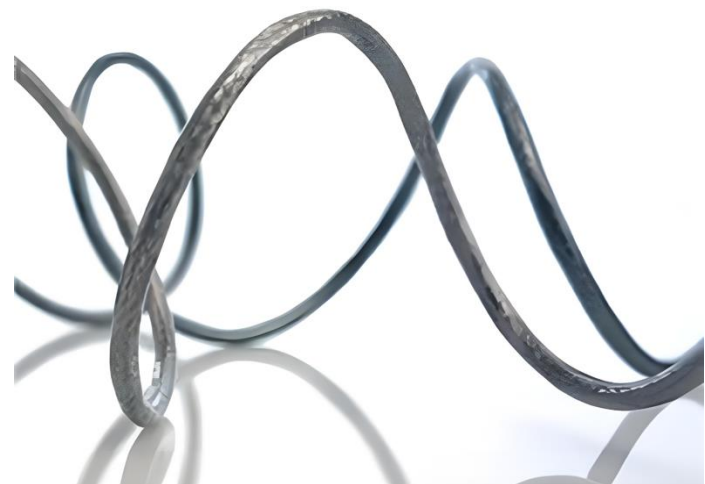
- 带有两个并行for指令和两个for指令的奇偶排序。

thread_count	1	2	3	4
两个omp parallel for指令	0.770	0.453	0.358	0.305
两个omp for指令	0.732	0.376	0.294	0.239

时间以秒为单位



循环的调度



schedule子句

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>
double f(int i) {
    int j, start = i * (i + 1) / 2, finish = start + i;
    double return_val = 0.0;
    for (j = start; j <= finish; j++) {
        return_val += sin(j);
    }
    return return_val;
}
int main(int argc, char* argv[]) {
    int thread_count = 4;
    int n = 20000;
    double start, finish;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    start = omp_get_wtime();
```

```
    double sum = 0.0;
    #pragma omp parallel num_threads(thread_count)
    reduction (+: sum )
    {
        int rank = omp_get_thread_num();
        #pragma omp for //schedule(static, 1)
        for (int i = 0; i <= n; i++)
        {
            sum += f(i);
            //printf("iteration %d is assigned to thread %d\n",
i, rank);
        }
    }
    finish = omp_get_wtime();
    printf("Elapsed time = %f seconds\n", finish - start);
    printf("sum = %f\n", sum);
    return 0;
}
```

结果

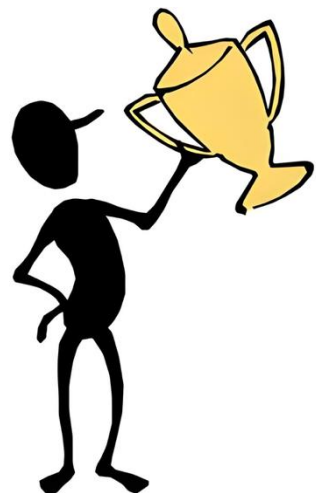
- $n = 10,000$
- 一个线程
 - ▶ 运行时间 = 3.67秒
- 两个线程
 - ▶ 默认分配
 - ▶ 运行时间 = 2.76秒
 - ▶ 加速比 = 1.33

```
double f(int i) {  
    int j, start = i * (i + 1) / 2, finish = start + i;  
    double return_val = 0.0;  
    for (j = start; j <= finish; j++) {  
        return_val += sin(j);  
    }  
    return return_val;  
}
```

调用函数 $f(i)$ 会调用 $\sin$ 函数 i 次，执行 $f(2i)$ 需要的时间大约是执行 $f(i)$ 花费时间的两倍。

循环划分

- 迭代一次一个，以轮流的方式分配给线程。
- 两个线程
 - ▶ 循环分配
 - ▶ 运行时间 = 1.84秒
 - ▶ 加速比 = 1.99



线程	迭代
0	$0, t, 2t, \dots$
1	$1, t+1, 2t+1, \dots$
$\vdots$	$\vdots$
$t-1$	$t-1, 2t-1, 3t-1, \dots$

schedule子句

■ 默认调度:

```
sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum)
    for (i = 0; i <= n; i++)
        sum += f(i);
```

■ 循环调度:

```
sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum) schedule(static,1)
    for (i = 0; i <= n; i++)
        sum += f(i);
```


schedule (type, chunksize)

■ type (调度类型) 可以是：

- ▶ static：迭代可以在循环执行前分配给线程。
- ▶ dynamic或guided：迭代是在循环执行时分配给线程的，因此在一个线程完成其当前的迭代集后，它可以向运行时系统请求更多的迭代。
- ▶ auto：编译器和 / 或运行时系统决定调度方式。
- ▶ runtime：调度是在运行时根据环境变量确定的。

■ chunksize (块大小) 是一个正整数。

静态调度类型

- 12个迭代0, 1,...,11, 3个线程。

schedule (static, 1)

线程0: 0, 3, 6, 9

线程1: 1, 4, 7, 10

线程2: 2, 5, 8, 11

schedule (static, 2)

线程0: 0, 1, 6, 7

线程1: 2, 3, 8, 9

线程2: 4, 5, 10, 11

schedule (static, 4)

线程0: 0, 1, 2, 3

线程1: 4, 5, 6, 7

线程2: 8, 9, 10, 11

动态调度类型

- 迭代同样会被拆分为若干个块，每个块包含连续的`chunksize`个迭代。
- 每个线程执行一个块，当某个线程完成一个块的执行后，会向运行时系统请求下一个块。
- 此过程会持续进行，直至所有迭代执行完毕。
- `chunksize`参数可省略；若省略该参数，则默认使用1作为块大小。

guided调度类型

- 每个线程同样执行一个数据块；当某个线程完成一个数据块后，会请求下一个数据块。
- 但在guided调度模式下，随着数据块不断完成，新分配数据块的大小会逐渐减小。
- 若未指定`chunksize`，则数据块的大小会逐步减小至 1。
- 若指定了`chunksize`，则数据块大小会递减至该指定值，例外情况是：最后一个数据块的大小可能小于`chunksize`。

guided调度类型

- 两个线程，guided调度对梯形法则迭代1 ~ 9999的赋值。

线程	块	块大小	剩下的迭代
0	1 ~ 5 000	5 000	4 999
1	5 001 ~ 7 500	2 500	2 499
1	7 501 ~ 8 750	1 250	1 249
1	8 751 ~ 9 375	625	624
0	9 376 ~ 9 687	312	312
1	9 688 ~ 9 843	156	156
0	9 844 ~ 9 921	78	78
1	9 922 ~ 9 960	39	39
1	9 961 ~ 9 980	20	19
1	9 981 ~ 9 990	10	9
1	9 991 ~ 9 995	5	4
0	9 996 ~ 9 997	2	2
1	9 998 ~ 9 998	1	1
0	9 999 ~ 9 999	1	0

runtime调度类型

- 系统使用环境变量OMP_SCHEDULE来决定在运行时如何调度循环。
 -
- OMP_SCHEDULE环境变量可以采用任何可用于static、dynamic或guided调度的值。

生产者和消费者



队列

- 可以将其类比为超市里排队结账的顾客队伍。
- 它是多线程应用中非常常用的一种数据结构。
- 例如：假设有若干个生产者线程和若干个消费者线程。
 - ▶ 生产者线程可以产生数据请求。
 - ▶ 消费者线程则可以通过查找或生成所需数据来处理 / 消费这些请求。

消息传递

- 每个线程都可以拥有一个共享消息队列，当一个线程想要向另一个线程发送消息时，它可以将消息入队到目标线程的队列中。
- 线程可以通过将其消息队列队首的消息出队来接收消息。

```
for (sent_msgs = 0; sent_msgs < send_max; sent_msgs++) {  
    Send_msg();  
    Try_receive();  
}
```

```
while (!Done())  
    Try_receive();
```

发送消息和接收消息

■ Send_msg

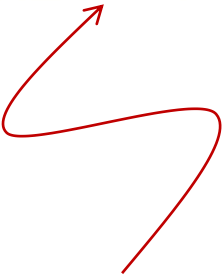
```
msg = random();  
dest = random() % thread_count;  
# pragma omp critical  
  Enqueue(queue, dest, my_rank, msg);
```

■ Try_receive

```
queue_size = enqueued - dequeued;  
if (queue_size == 0) return;  
else if (queue_size == 1)  
#   pragma omp critical  
    Dequeue(queue, &src, &msg);  
else  
    Dequeue(queue, &src, &msg);  
  Print_message(src, msg);
```

终止检测

```
queue_size = enqueued - dequeued;  
if (queue_size == 0 && done_sending == thread_count)  
    return TRUE;  
else  
    return FALSE;
```



每个线程在完成其for循环后都会增加这个计数器。

开始

- 当程序开始执行时，主线程会获取命令行参数，并分配一个消息队列数组：每个线程对应一个消息队列。
- 这个数组需要在线程间共享，因为任意线程都可以向其他任意线程发送消息，因此任意线程都可以向任意队列中入队消息。
- 可能会出现部分线程比其他线程更早完成队列分配的情况。
- 需要一个显式栅栏，使得当一个线程执行到栅栏时，它会阻塞，直到线程组内的所有线程都到达栅栏。
- 所有线程都到达栅栏后，线程组内的全部线程才能继续执行。

pragma omp barrier

atomic指令

- 与critical指令不同，它只能保护由单个C赋值语句构成的临界区。

pragma omp atomic

- 此外，该语句必须是以下形式之一：

`x <op>= <expression>;`

`x++;`

`++x;`

`x--;`

`--x;`

- 这里<op> 可以是以下二元运算符之一：

`+, *, -, /, &, ^, |, <<, >>`

atomic指令

- 许多处理器都提供一条特殊的加载-修改-存储指令。
- 对于仅执行加载-修改-存储操作的临界区，使用这条特殊指令进行保护，会比用那些保护更通用临界区的结构高效得多。

临界区

- OpenMP支持为临界区指令添加名称：

pragma omp critical (name)

- 当这样做时，使用不同名称的critical指令保护的两个代码块，可以同时执行。
- 但是，这些名称是在编译期间确定的，而我们需要为每个线程的队列分别设置不同的临界区。

锁

- 锁由一种数据结构和若干函数组成，这些函数允许程序员在临界区中显式地实现互斥。



/* 由一个线程执行 */

初始化锁的数据结构；

...

/* 由多个线程执行 */

尝试锁定或设置锁的数据结构；

临界区；

解锁或取消设置锁数据结构；

...

/* 由一个线程执行 */

销毁锁的数据结构；

在消息传递程序中使用锁

```
# pragma omp critical
  /* q_p = msg_queues[dest] */
  Enqueue(q_p, my_rank, mesg);
```

```
/* q_p = msg_queues[dest] */
omp_set_lock(&q_p→lock);
Enqueue(q_p, my_rank, mesg);
omp_unset_lock(&q_p→lock);
```

在消息传递程序中使用锁

```
# pragma omp critical
/* q_p = msg_queues[my_rank] */
Dequeue(q_p, &src, &msg);
```

```
/* q_p = msg_queues[my_rank] */
omp_set_lock(&q_p->lock);
Dequeue(q_p, &src, &msg);
omp_unset_lock(&q_p->lock);
```

注意事项

- 你不应该在同一个临界区中混用不同类型的互斥机制。
- 互斥构造不保证公平性。
- 嵌套使用互斥构造可能会带来危险。

缓存、缓存一致性与伪共享

- 回想一下，芯片设计师在处理器中增加了速度相对较快的存储器块，称为高速缓存（cache memory）。
- 高速缓存的使用会对共享内存产生巨大影响。
- 当处理器核心试图更新一个不在高速缓存中的变量时，就会发生写缺失，此时它必须访问主存。

缓存一致性

- CPU缓存是由系统硬件管理的，程序员不能直接控制它们。

y0由核心0私有

y1和z1由核心1私有

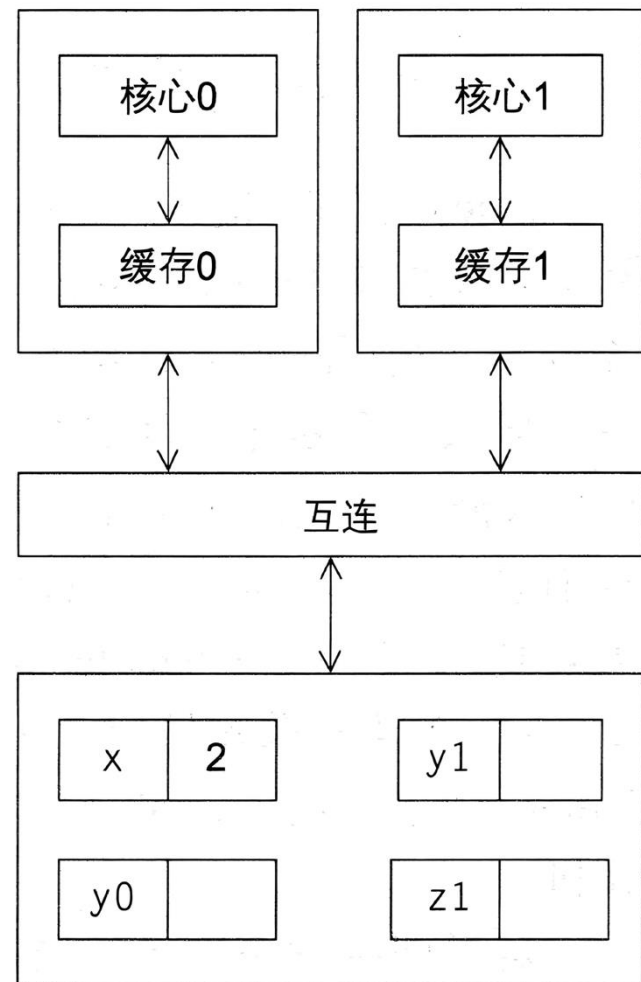
x = 2; /*共享变量*/

时刻	核心 0	核心 1
0	y0 = x;	y1 = 3*x;
1	x = 7;	不涉及 x 的语句
2	不涉及 x 的语句	z1 = 4*x;

y0 = 2

y1 = 6

z1 = ???



具有两个核心和两个缓存的共享内存系统

矩阵-向量乘法

$$y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots + a_{i,n-1}x_{n-1}$$

a_{00}	a_{01}	$\cdots$	$a_{0,n-1}$		y_0
a_{10}	a_{11}	$\cdots$	$a_{1,n-1}$		y_1
$\vdots$	$\vdots$		$\vdots$		$\vdots$
a_{i0}	a_{i1}	$\cdots$	$a_{i,n-1}$	x_0	$y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots + a_{i,n-1}x_{n-1}$
$\vdots$	$\vdots$		$\vdots$	x_1	$\vdots$
$a_{m-1,0}$	$a_{m-1,1}$	$\cdots$	$a_{m-1,n-1}$	$\vdots$	y_{m-1}
				x_{n-1}	

```

for (i = 0; i < m; i++) {
    y[i] = 0.0;
    for (j = 0; j < n; j++)
        y[i] += A[i][j]*x[j];
}

```

OpenMP矩阵-向量乘法

```
# pragma omp parallel for num_threads(thread_count) \
    default(none) private(i, j) shared(A, x, y, m, n)
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    for (j = 0; j < n; j++)
        y[i] += A[i][j]*x[j];
}
```

OpenMP矩阵-向量乘法

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
const int m = 8000000;
const int n = 8;
double A[m][n];
double x[n];
double y[m];
int main(void) {
    int i, j;
    double start, finish;
    printf("Matrix %d×%d\n", m, n);
    for (int thread_count = 1; thread_count <= 8;
        thread_count *= 2) {
        start = omp_get_wtime();
        # pragma omp parallel for
```

```
num_threads(thread_count) private(i, j)
    for (i = 0; i < m; i++) {
        y[i] = 0.0;
        for (j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
    finish = omp_get_wtime();
    double elapsed = finish - start;
    printf("Elapsed time of %ld threads = %f
seconds\n", thread_count, elapsed);
    }
    return 0;
}
```


矩阵-向量乘法的运行时间和效率

线程	矩阵维度					
	8 000 000 × 8		8 000 × 8 000		8 × 8 000 000	
	时间	效率	时间	效率	时间	效率
1	0.322	1.000	0.264	1.000	0.333	1.000
2	0.219	0.735	0.189	0.698	0.300	0.555
4	0.141	0.571	0.119	0.555	0.303	0.275

(时间单位为秒)

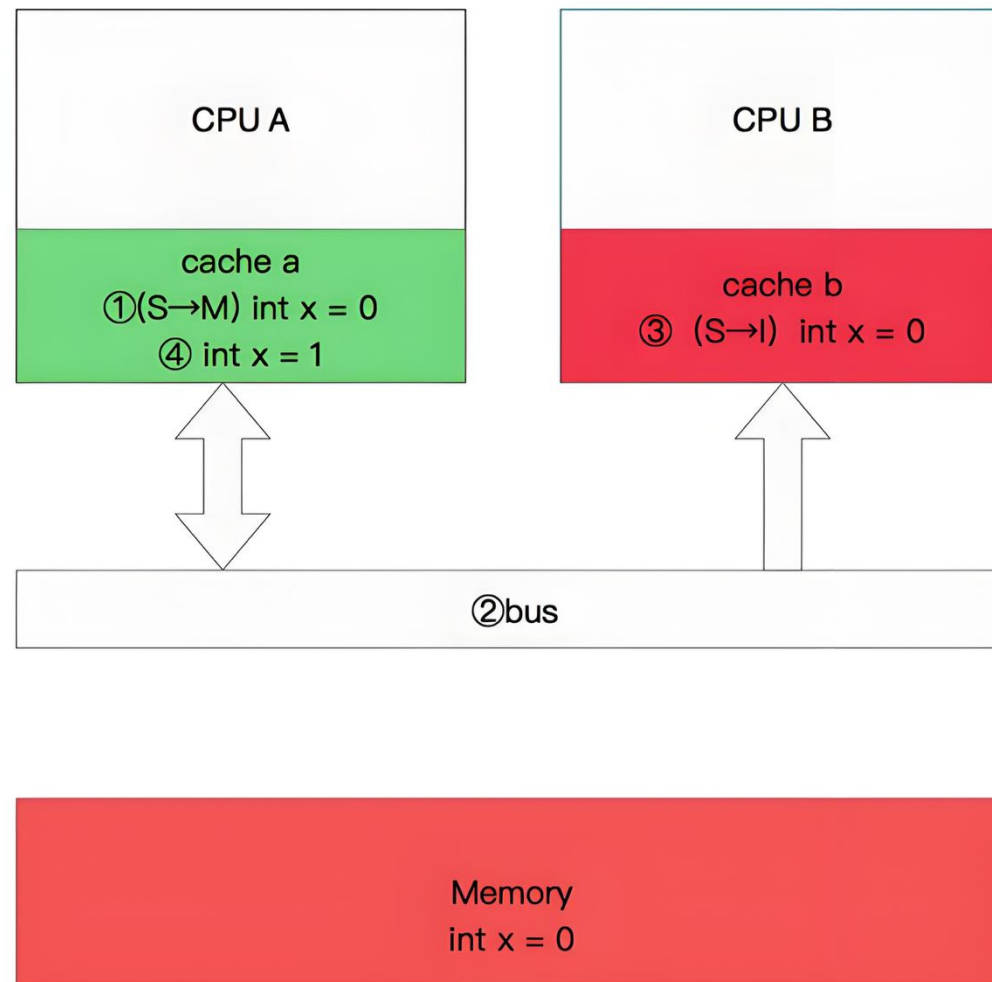
缓存伪共享

■ 缓存行 (Cache Line)

- ▶ CPU缓存与内存交换的最小单位，现代x86/ARM通常为64字节。
- ▶ 访问一个变量时，会把它所在的整个64字节块加载到缓存。

■ 缓存一致性协议

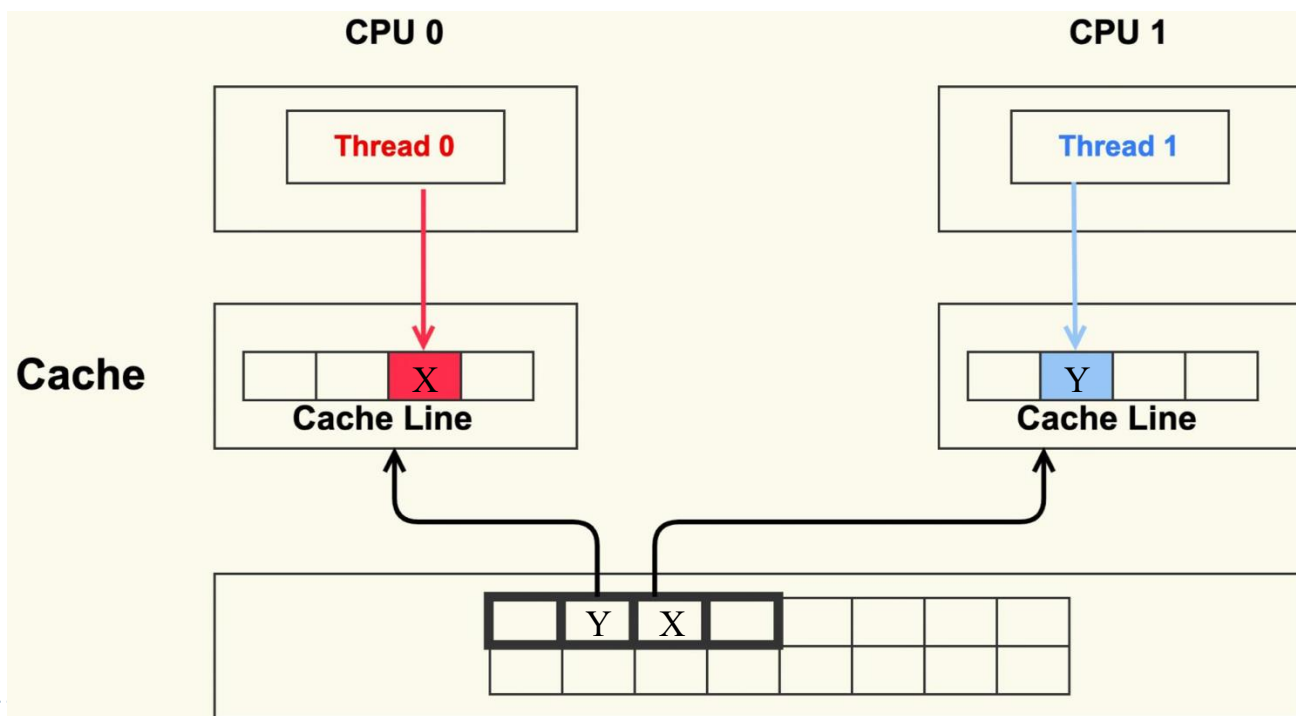
- ▶ 多核间通过缓存一致性协议保证缓存一致。
- ▶ 当一个核心修改缓存行中的任意一个变量，会将该行标记为Modified（脏），并向所有其他核心发送无效化（Invalidate）消息，强制它们的副本失效。



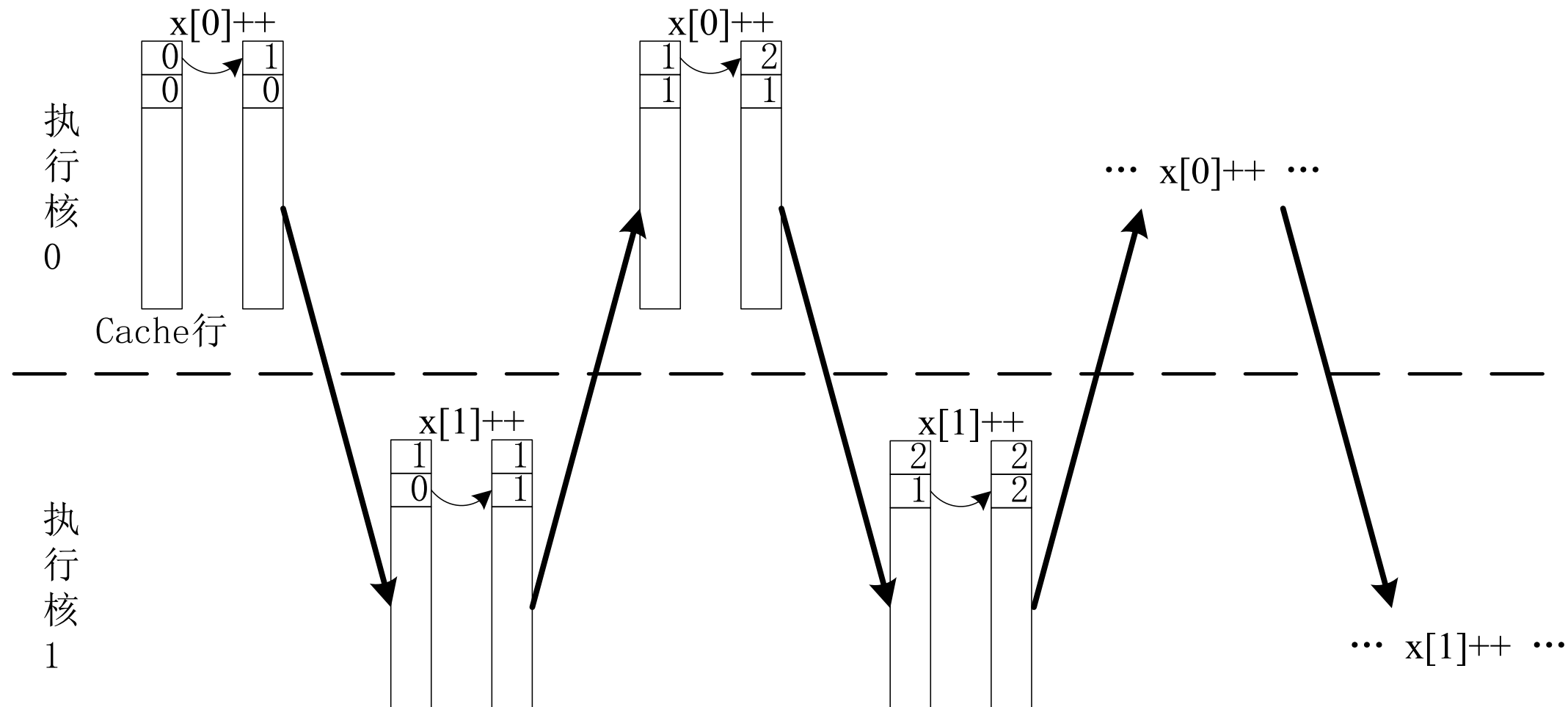
缓存伪共享

■ 伪共享的发生

- ▶ 线程0修改变量X，线程1修改变量Y。
- ▶ X、Y逻辑无关，但在内存中相邻，落入同一个64字节缓存行。
- ▶ P0修改X→整个缓存行失效→P1访问Y时必须重新从内存加载→性能雪崩。



伪共享引起的Cache行乒乓现象



基于数组的OpenMP全局求和

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
using namespace std;
int main(void){
    int k;
    int n = 1000000000;
    double factor;
    double sum;
    double my_sum[1000];
    double start, finish, elapsed;
    for (int thread_count = 1; thread_count <= 8;
        thread_count *= 2)
    {
        sum = 0.0;
        for (int i = 0; i < 1000; i++)
            my_sum[i] = 0.0;
        start = omp_get_wtime();
        #pragma omp parallel num_threads(thread_count)
        private(factor, k)
    {
```

```
        int rank = omp_get_thread_num();
        #pragma omp for
        for (k = 0; k < n; k++) {
            if (k % 2 == 0)
                factor = 1.0;
            else
                factor = -1.0;
            my_sum[rank] += factor / (2 * k + 1);
        }
    }
    finish = omp_get_wtime();
    elapsed = finish - start;
    for (int i = 0; i < 1000; i++)
        sum += my_sum[i];
    sum = 4.0 * sum;
    printf("thread_count = %d:\n", thread_count);
    printf("Multi-threaded estimate of pi = %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
}
```

线程安全

```
void Tokenize(  
    char* lines[]      /* in/out */,  
    int line_count     /* in      */,  
    int thread_count   /* in      */) {  
    int my_rank, i, j;  
    char *my_token;  
# pragma omp parallel num_threads(thread_count) \  
    default(none) private(my_rank, i, j, my_token) shared(lines, line_count)  
    {  
        my_rank = omp_get_thread_num();  
# pragma omp for schedule(static, 1)  
        for (i = 0; i < line_count; i++) {  
            printf("Thread %d > line %d = %s", my_rank, i, lines[i]);  
            j = 0;  
            my_token = strtok(lines[i], " \t\n");  
            while ( my_token != NULL ) {  
                printf("Thread %d > token %d = %s\n", my_rank, j, my_token);  
                my_token = strtok(NULL, " \t\n");  
                j++;  
            }  
        } /* for i */  
    } /* omp parallel */  
} /* Tokenize */
```

任务并行

■ #pragma omp task

- ▶ 专门解决递归、不规则循环、动态任务等传统并行指令（parallel for）无法高效处理的场景。
- ▶ 把一段代码创建成独立的可调度任务，交给线程池中的空闲线程自动执行，实现动态负载均衡。

任务并行

■ 适用场景

- ▶ 递归函数（如快速排序、树遍历）。
- ▶ 不规则循环（循环次数未知 / 迭代工作量不均）。
- ▶ 动态生成的子任务。

■ 执行规则

- ▶ 任务由当前线程组内的空闲线程抢占执行。
- ▶ 任务会延迟执行，直到线程空闲或遇到任务同步点。

■ 必须配合：`#pragma omp parallel`（创建线程池）。

示例

```
#include <iostream>
#include <omp.h>
int main() {
    #pragma omp parallel num_threads(4)
    {
        #pragma omp single //单线程生成任务,
        避免重复创建
        {
            std::cout << "主线程开始创建任务\n";
            #pragma omp task //创建任务1
            {
                printf("任务1由线程%d执行\n",
                    omp_get_thread_num());
            }
        }
    }
}
```

```
    }
    #pragma omp task //创建任务2
    {
        printf("任务2由线程%d执行\n",
            omp_get_thread_num());
    }
}
return 0;
}
```

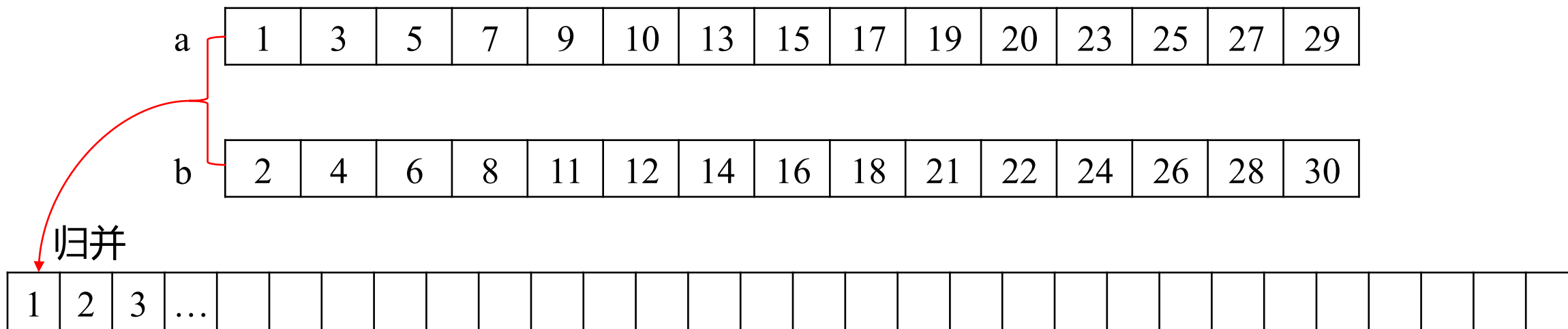
使用OpenMP任务计算斐波那契数

```
#include <iostream>
#include <omp.h>
using namespace std;
int fib(int n) {
    int i = 0;
    int j = 0;
    if (n < 2)
        return n;
    #pragma omp task shared(i) if (n > 20)
    i = fib(n - 1);
    #pragma omp task shared(j) if (n > 20)
    j = fib(n - 2);
    #pragma omp taskwait
    return i + j;
}
```

```
}
int main() {
    int res;
    double start, finish;
    start = omp_get_wtime();
    #pragma omp parallel num_threads(4)
    #pragma omp single
    res = fib(30);
    finish = omp_get_wtime();
    cout << "fib(10) = " << res << endl;
    cout << "Elapsed time = " << finish - start
    << " seconds" << endl;
    return 0;
}
```

归并

- 将两个已排序的子数组，合并成一个整体有序的数组。
- 串行算法
 - 双指针遍历：用指针 i 、 j 分别指向两个子数组的起始位置。
 - 比较与复制：每次取两指针指向元素中较小者放入目标数组，并移动对应指针。



串行归并程序

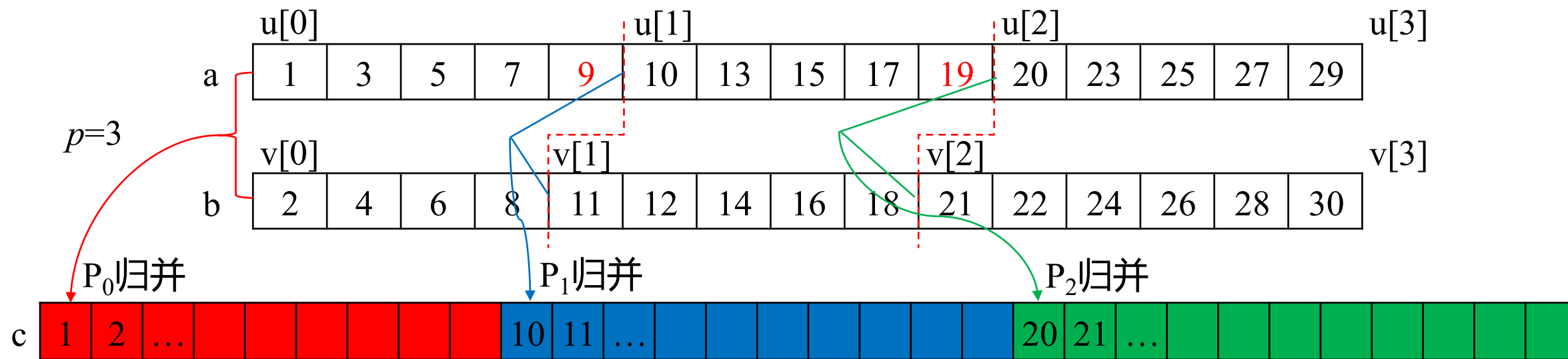
```
#include <iostream>
#include <algorithm>
int m = 1e8;
int n = 1e8;
using namespace std;
int main(void) {
    double* a = new double[m];
    double* b = new double[n];
    double* c = new double[m + n];
    struct timespec start, finish;
    int i, j, k;
    srand(time(NULL));
    a[0] = rand() * 1.0 /
    RAND_MAX;
    for (i = 1; i < m; i++)
        a[i] = a[i - 1] + rand() * 1.0 /
        RAND_MAX;
    b[0] = rand() * 1.0 /
```

```
RAND_MAX;
    for (j = 1; j < n; j++)
        b[j] = b[j - 1] + rand() * 1.0 /
        RAND_MAX;
    timespec_get(&start,
    TIME_UTC);
    i = j = k = 0;
    while (1)
    {
        if (a[i] < b[j])
        {
            c[k++] = a[i++];
            if (i >= m)
                break;
        }
        else
        {
            c[k++] = b[j++];
```

```
        if (j >= n)
            break;
        }
    }
    while (i < m)
        c[k++] = a[i++];
    while (j < n)
        c[k++] = b[j++];
    timespec_get(&finish,
    TIME_UTC);
    cout << "serial time = " <<
    finish.tv_sec - start.tv_sec +
    (finish.tv_nsec - start.tv_nsec) /
    1e9 << endl;
    delete[] a;
    delete[] b;
    delete[] c;
}
```

归并

```
u[0]=v[0]=0;
u[p]=m;
v[p]=n;
for (i=1; i<p; i++) //并行求解
    u[i]=i*m/p;
    v[i]=rank(b, b+n, a[u[i]-1]); //求数组b中 $\leq a[u[i]-1]$ 的元素个数
for (i=0; i<p; i++) //并行求解
    merge(a+u[i], a+u[i+1], b+v[i], b+v[i+1], c+u[i]+v[i]); //归并
```



并行归并程序

```
#include <iostream>
#include <algorithm>
#include <omp.h>
int m = 1e8;
int n = 1e8;
using namespace std;
int main(void) {
    double* a = new double[m];
    double* b = new double[n];
    double* c = new double[m + n];
    double* d = new double[m + n];
    srand(time(NULL));
    a[0] = rand() * 1.0 / RAND_MAX;
    for (int i = 1; i < m; i++)
        a[i] = a[i - 1] + rand() * 1.0 /
            RAND_MAX;
    b[0] = rand() * 1.0 / RAND_MAX;
    for (int i = 1; i < n; i++)
        b[i] = b[i - 1] + rand() * 1.0 /
            RAND_MAX;
    double start, finish, parallel_time,
    serial_time;
```

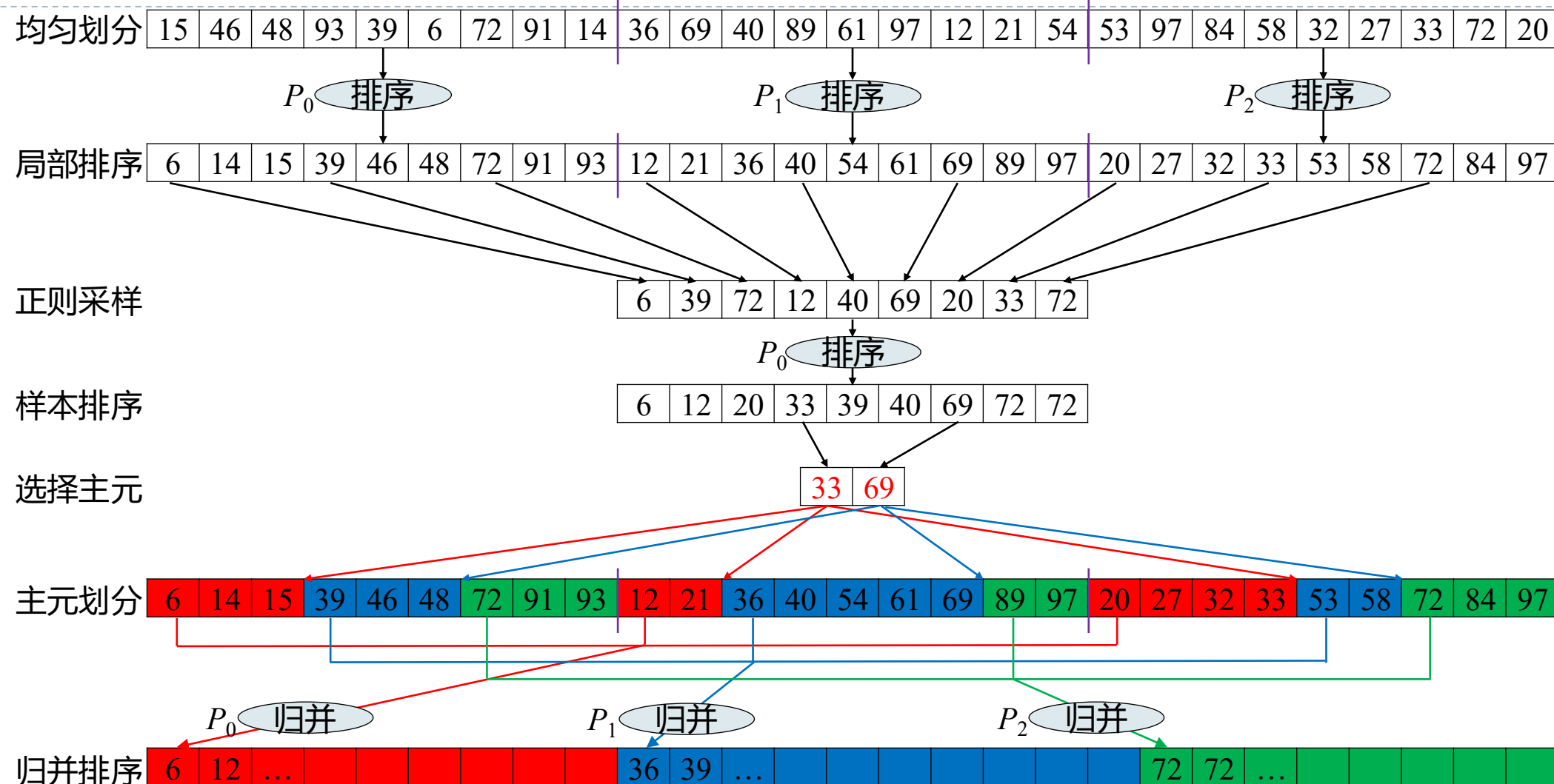
```
start = omp_get_wtime();
int p = 8;
int* u = new int[p + 1];
int* v = new int[p + 1];
u[0] = v[0] = 0;
u[p] = m;
v[p] = n;
#pragma omp parallel num_threads(p)
{
    #pragma omp for
    for (int i = 1; i < p; i++)
    {
        u[i] = i * m / p;
        v[i] = upper_bound(b, b + n, a[u[i] - 1])
            - b;
    }
    #pragma omp for
    for (int i = 0; i < p; i++)
        merge(a + u[i], a + u[i + 1], b + v[i], b +
            v[i + 1], c + u[i] + v[i]);
}
finish = omp_get_wtime();
```

```
parallel_time = finish - start;
cout << "parallel time with " << p << "
threads = " << parallel_time << endl;
start = omp_get_wtime();
merge(a, a + m, b, b + n, d);
finish = omp_get_wtime();
serial_time = finish - start;
cout << "serial time = " << serial_time
<< endl;
cout << "speedup = " << serial_time /
parallel_time << endl;
for (int i = 0; i < m + n; i++)
    if (c[i] != d[i])
        cout << "error" << endl;
delete[] a;
delete[] b;
delete[] c;
delete[] d;
delete[] u;
delete[] v;
}
```

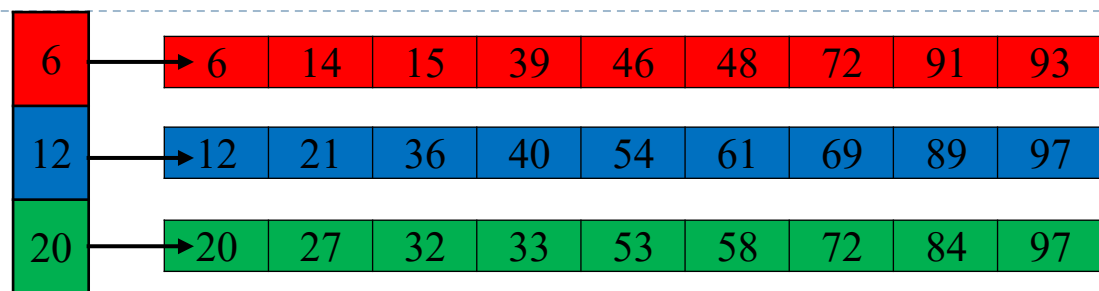
PSRS排序算法

- 均匀划分: n 个元素均匀地划分成 p 段, 每台处理器有 n/p 个元素。
- 局部排序: 各处理器利用串行排序算法, 排序 n/p 个数。
- 正则采样: 每台处理器各从自己的有序段中选取 p 个样本元素 (各段的第0、 n/p^2 、 $2n/p^2$ 、...个元素)。
- 样本排序: 用一台处理器将所有 p^2 个样本元素用串行排序算法排序之
- 选择主元: 用一台处理器从 p^2 个样本元素中选取 $p-1$ 个主元 (第 p 、 $2p$ 、...个元素)。
- 主元划分: 各处理器按主元将各自的有序段划分成 p 段。
- 归并排序: 各处理器对各段施行归并排序。

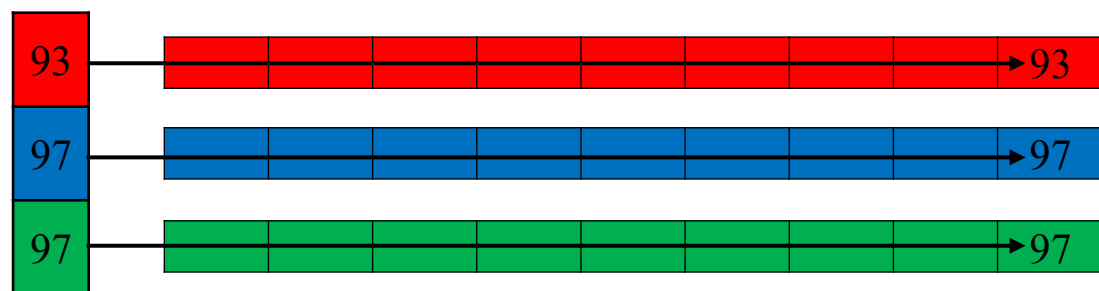
PSRS排序示例 ($n=27, p=3$)



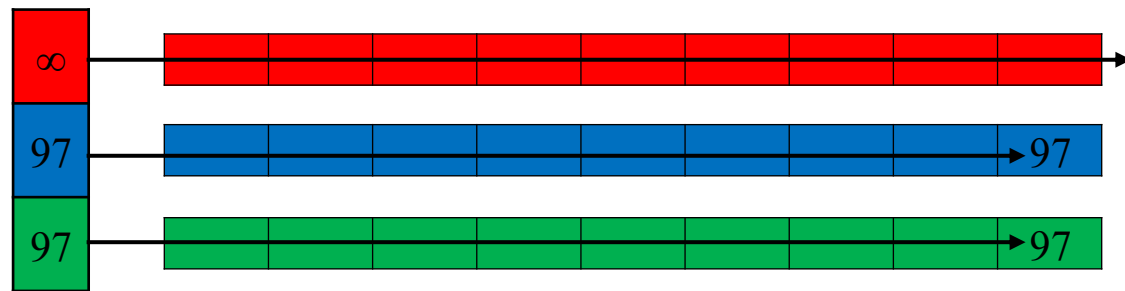
p 路归并



head



head



head

小结

- OpenMP是面向共享内存系统编程的一种标准。
- OpenMP既使用专用函数，也使用被称为pragma的预处理器指令。
- OpenMP程序启动的是多个线程，而非多个进程。
- 许多OpenMP指令都可以通过子句进行修饰。

小结

- 开发共享内存程序的一个主要问题是可能出现的竞争条件。
- OpenMP 提供了多种机制来确保临界区中的互斥：
 - ▶ critical指令
 - ▶ 命名critical指令
 - ▶ Atomic指令
 - ▶ 简单锁

小结

- 在默认情况下，大多数系统都使用块状划分的方式来划分并行化的for循环中的迭代。
- OpenMP提供了多种调度选项。
- 在OpenMP中，变量的作用域是指可以访问该变量的线程集合。

小结

- 归约是一种计算，将同一个归约运算符重复应用于一连串的操作数，以得到单一的结果。